Trees

[CSE2010] Data Structures

Department of Data Science

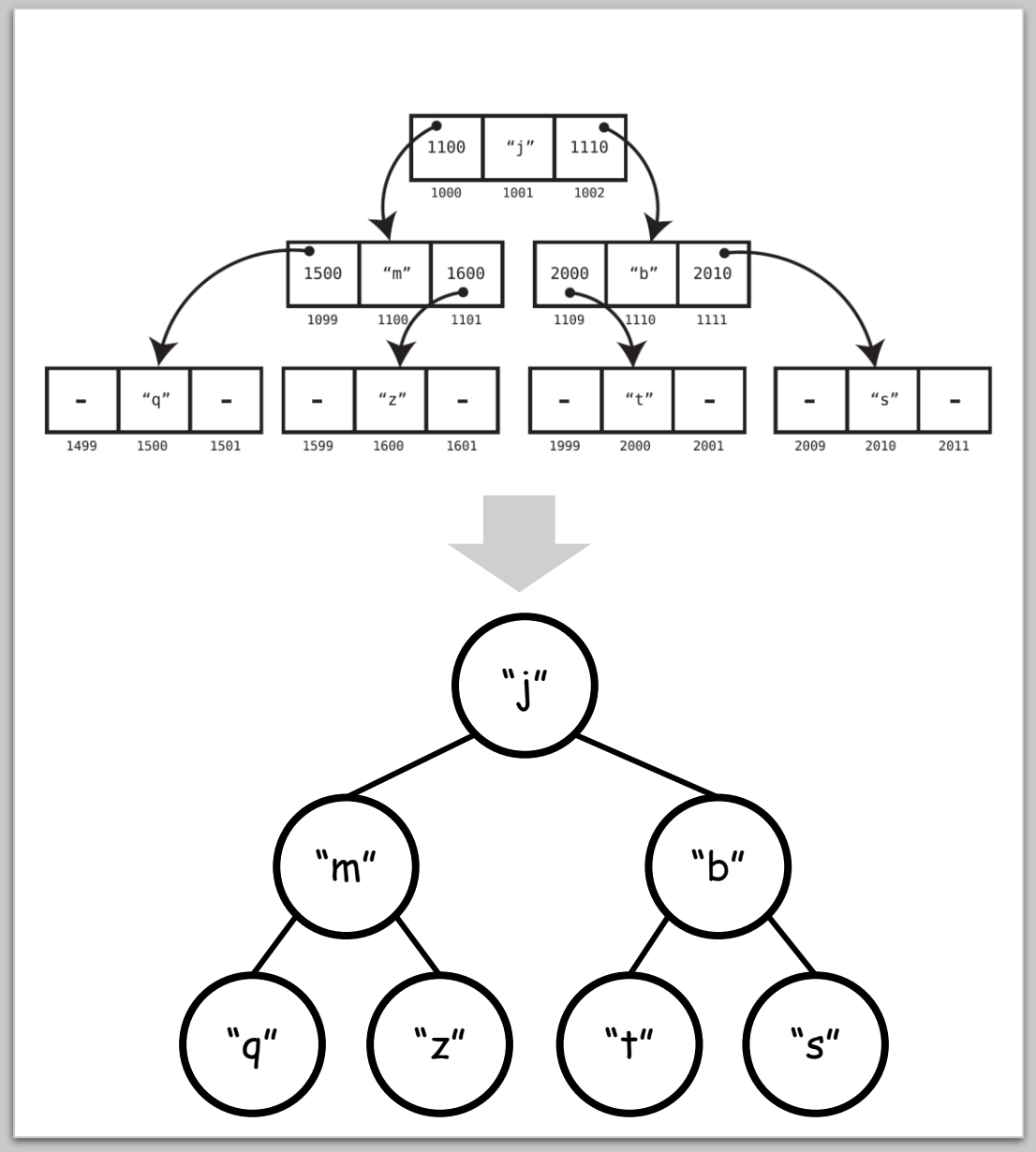
Tree

Tree is a collection of nodes connected by edges without cycles

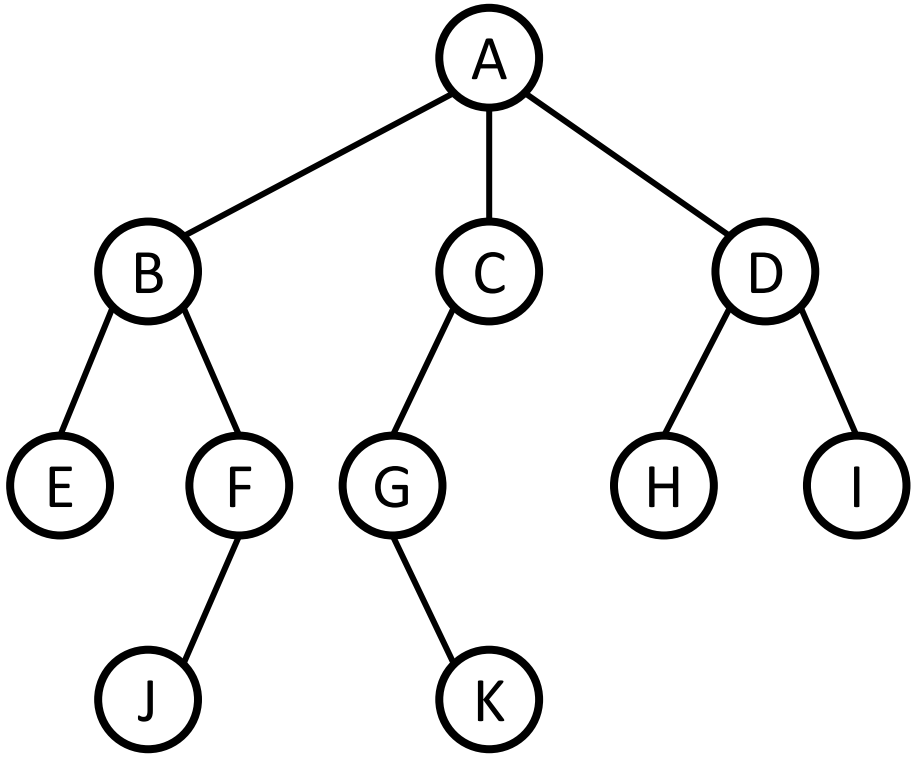
Trees

- Tree is a node-based data structure where each node can have links to *multiple* nodes.

- Represent trees visually without showing all the memory addresses.

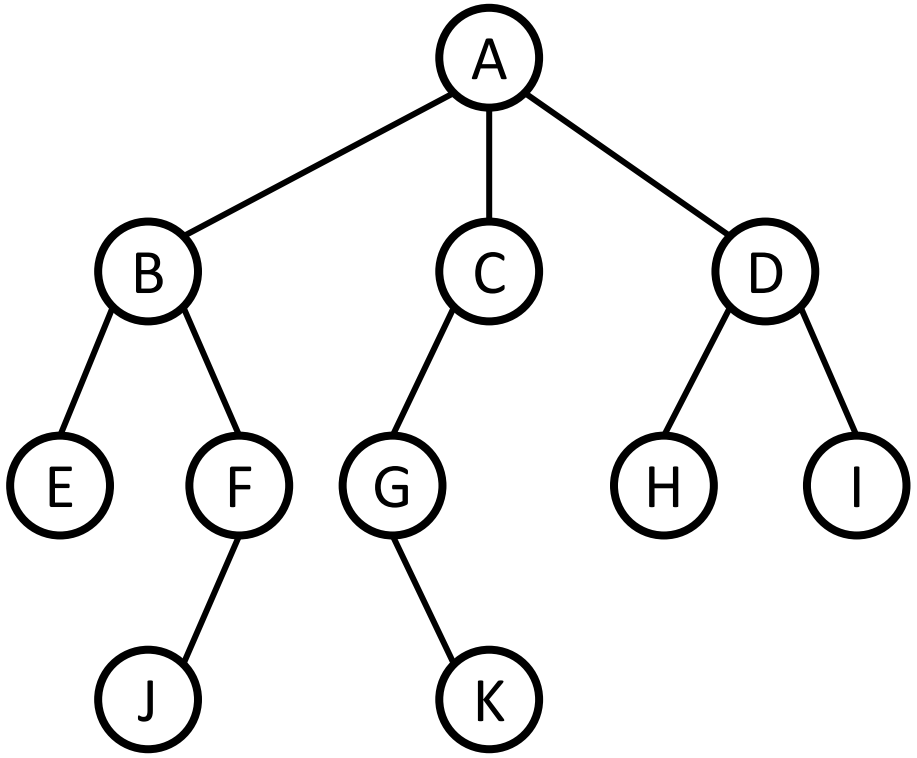


Terminologies



- **Root** is the node with no parents. There can be at most one root node in a tree (node A).
- **Edge** is the link from parent to child.
- P is **parent** of Q if there is an edge from P to Q. Q is called a **child** of P.
  - A is the parent of B, C, D.
- Children of same parents are called **siblings**.
  - B, C, D are siblings of A.
- A node with no children is **leaf** (E, J, K, H, and I).
- A node is **internal** if it has one or more children (A, B, C, D, F, G).
- P is **ancestor** of Q if there exists a path from root to Q and P appears on the path. Q is called a **descendant** of P.
  - A, C, G are the ancestors of K.

Terminologies



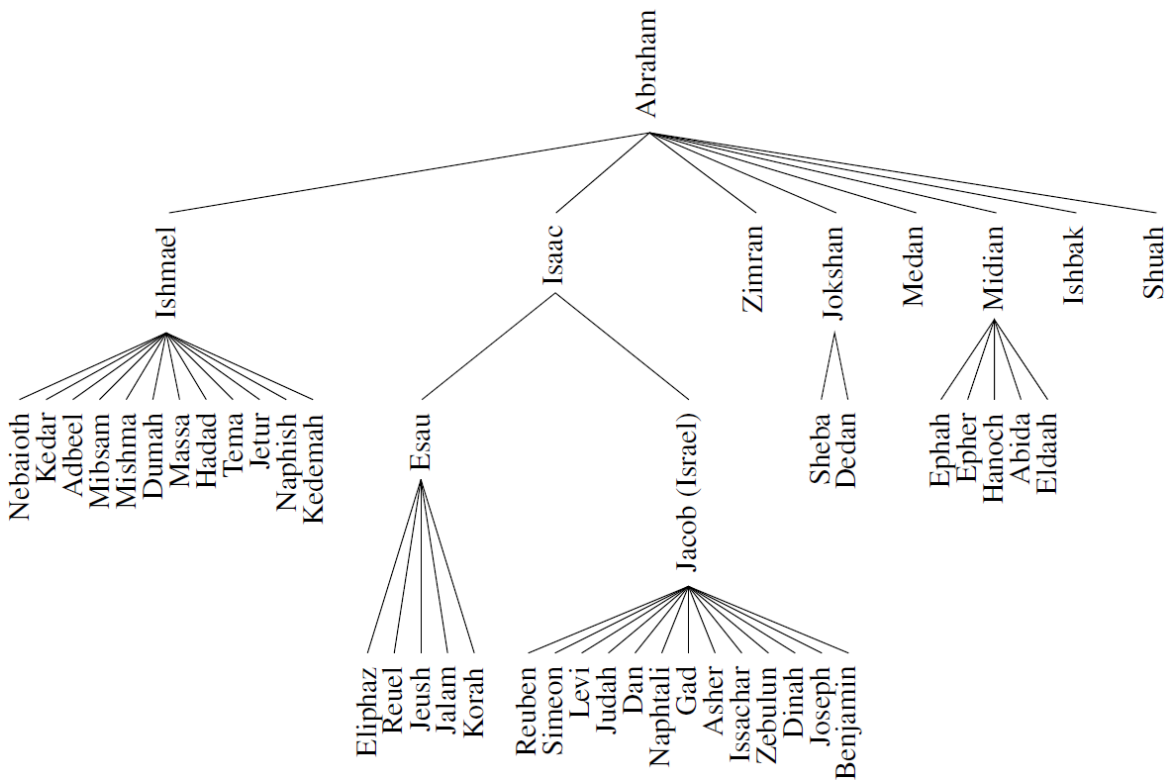
- **Subtree** of the tree rooted a node v is the tree consisting of all the descendants of v in T, including v itself.
  - The subtree rooted at D consists of D, H, I.
- **Path** between two nodes is a sequence of nodes $n_1, n_2, \ldots, n_k$ such that n_i is a parent of n_{i+1}. **Length of a path** is the number of edges on the path.
  - length of path A-C-G is 2.
- **Depth** (**level**) of a node is the length of the path from root to node. The root node is at level zero
  - B, C, D are the same level. The depth of G is 2.

Properties of Tree

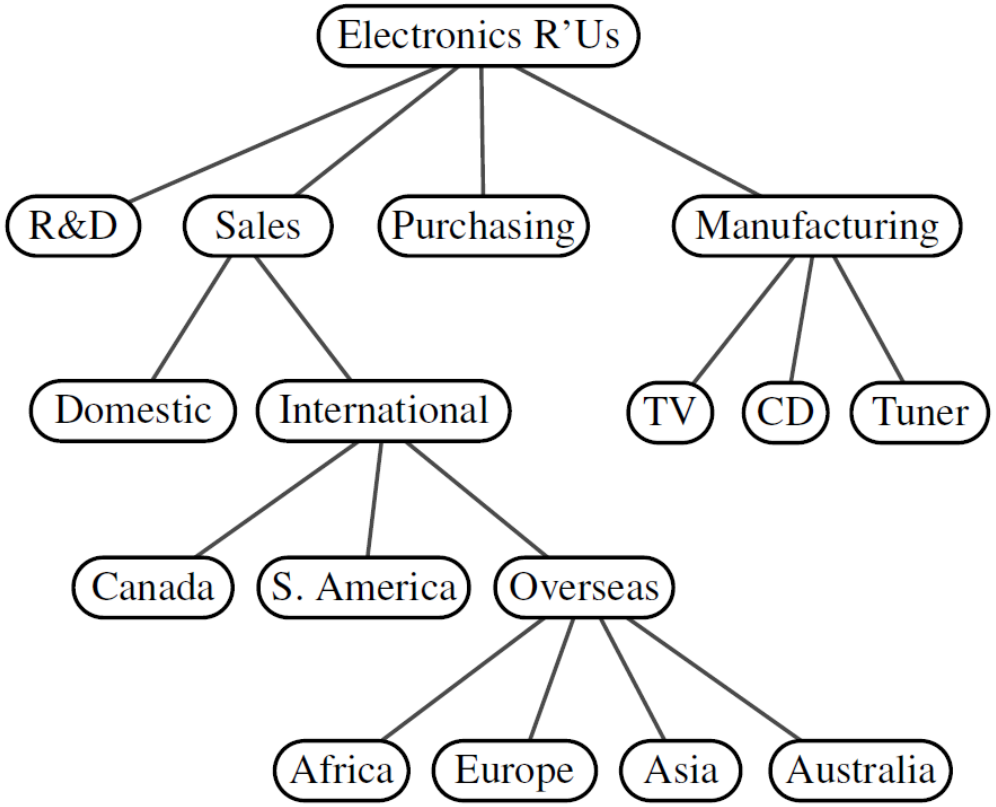
- Can a tree be empty with no nodes?
- For any node x, is there exactly a single path from the root to x?
- How many edges are in a tree with n nodes?

Common Usages of Trees

- Family tree

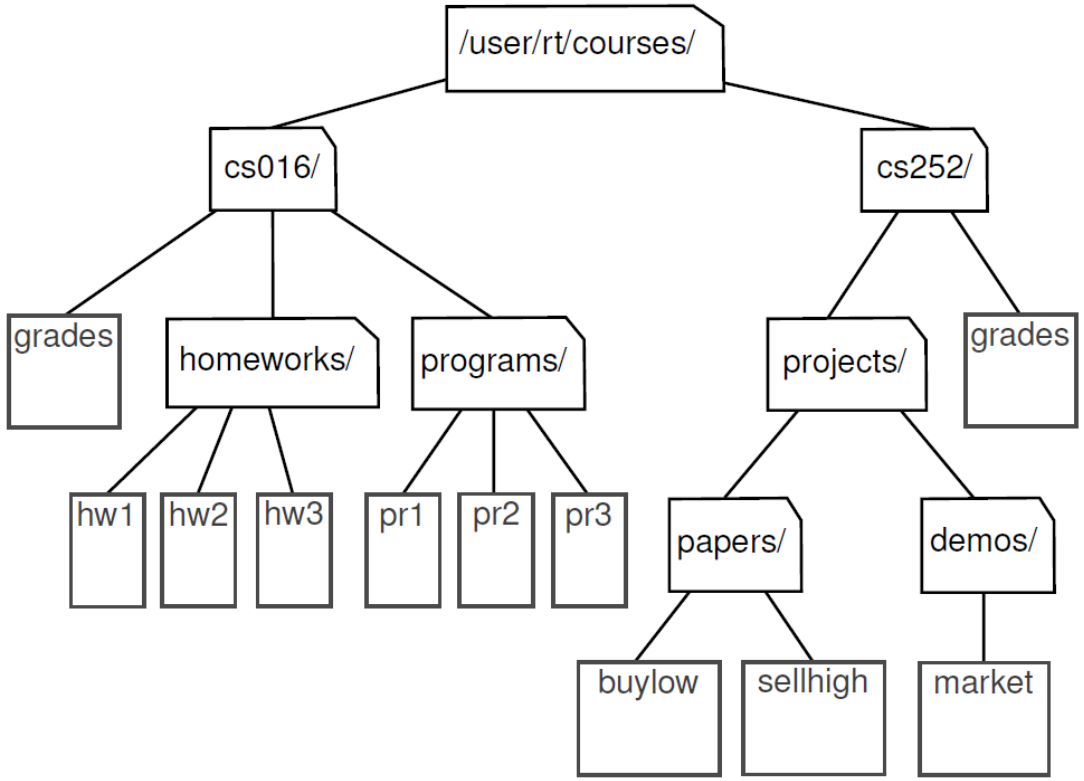


- Organization of corporation

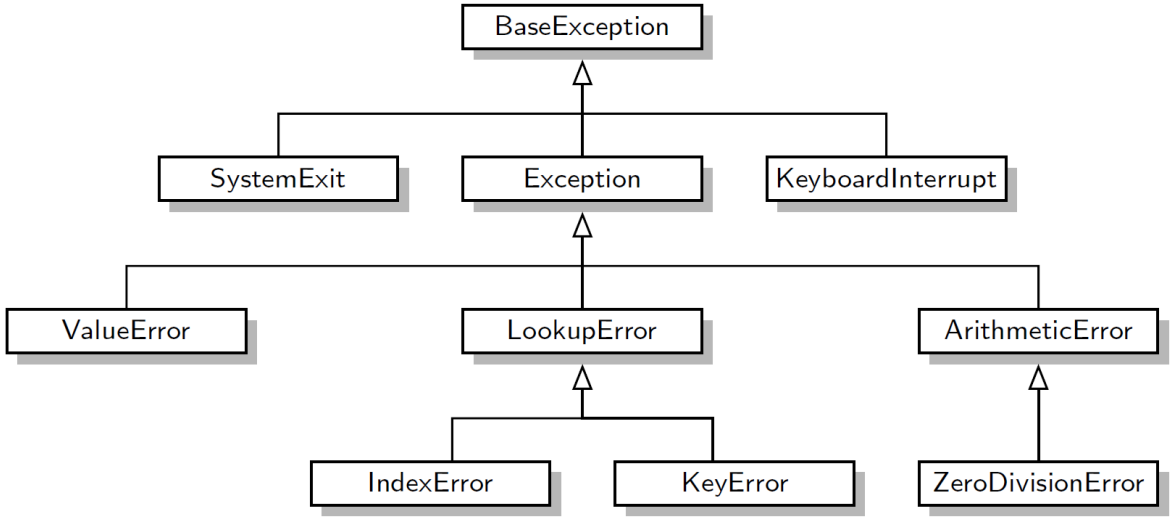


Common Usages of Trees

- File system

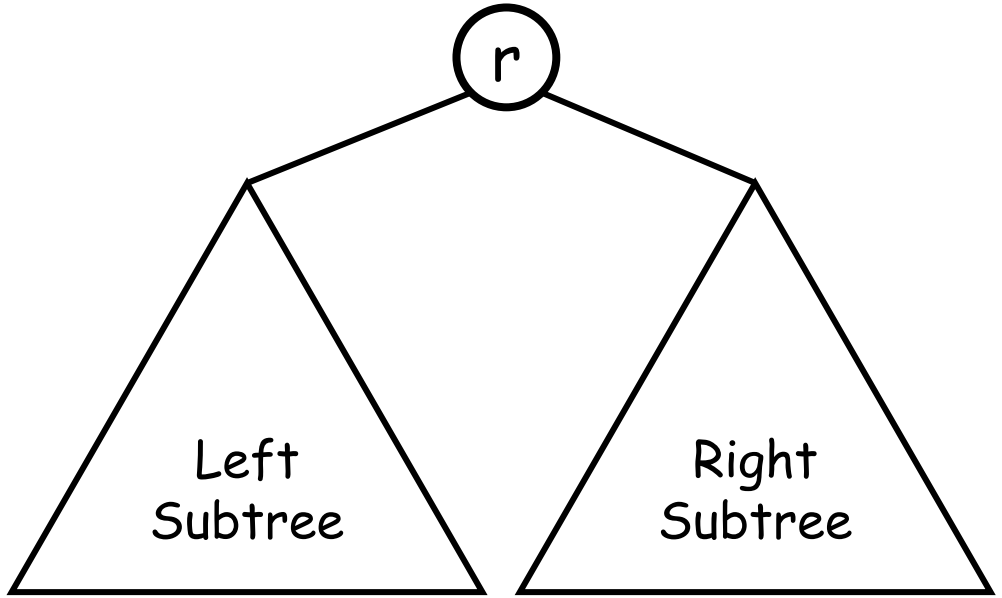


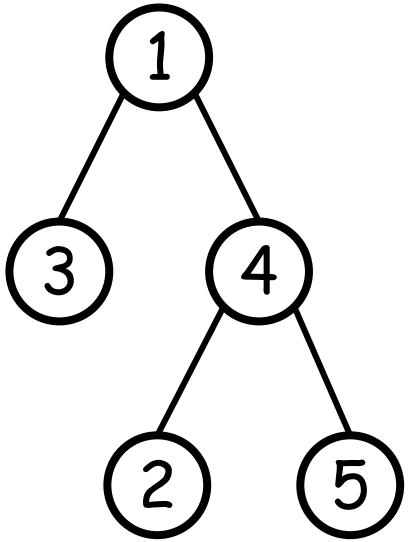
- Inheritance relations between classes



Binary Trees

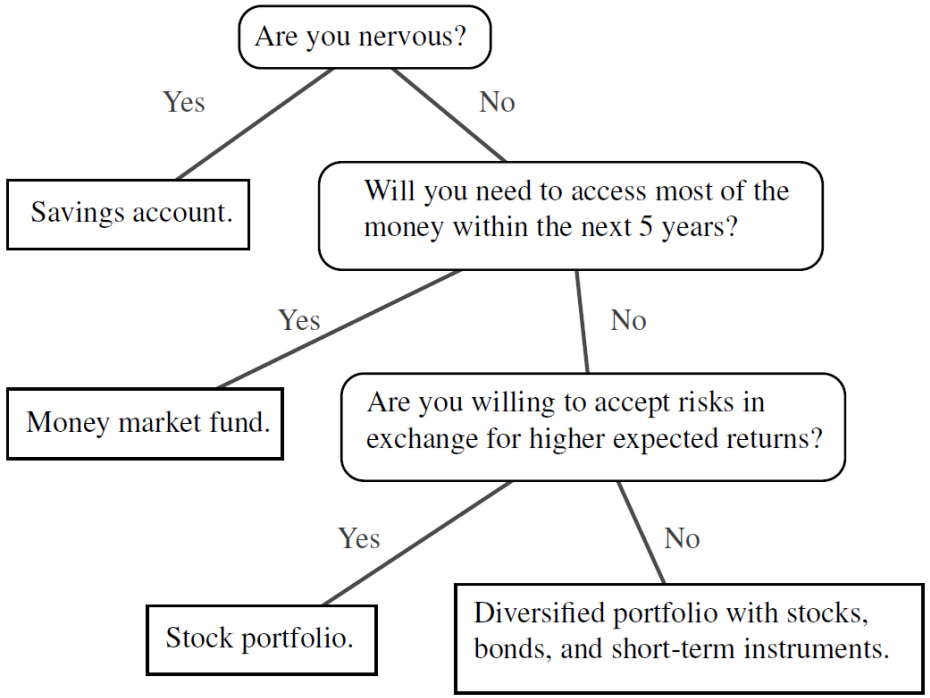
- **Binary tree** is a tree in which each node has 0, 1, or 2 children.
- Each child is labeled as being either a **left child** and a **right child**.
- Binary tree is either empty or consists of the root, the left subtree, and the right subtree.



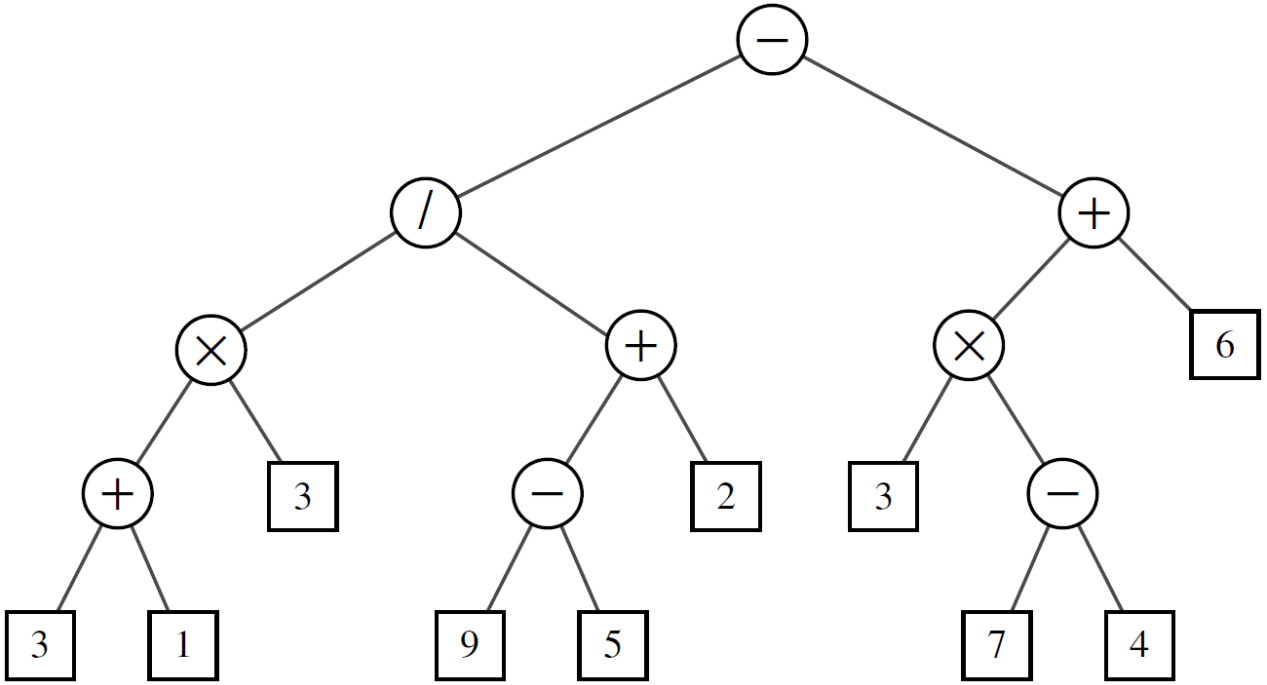


Common Usages of Binary Trees

- Decision tree providing investment advice

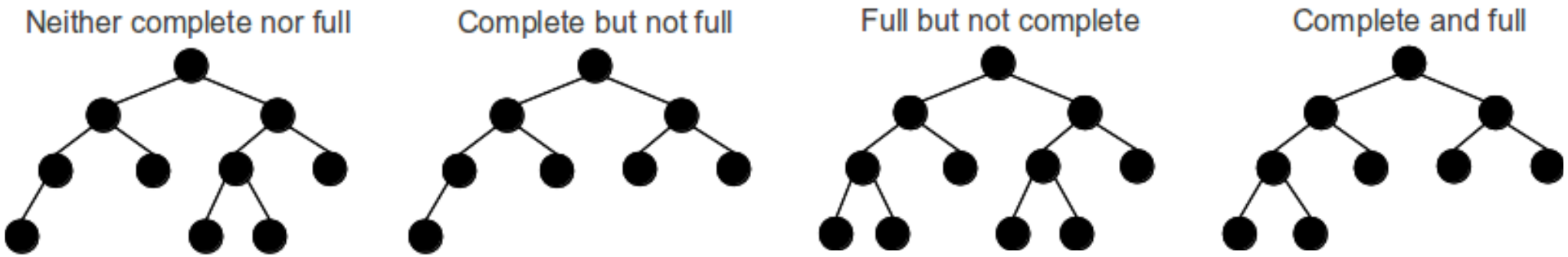


- Tree representing arithmetic expression
  ((((3+1) ×3)/((9-5)+2))-((3 ×(7-4))+6))



Binary Trees

- **Full binary tree** is a binary tree in which every node has 0 or 2 children.
- **Complete binary tree** is a binary tree in which every level except the last is completely filled and the last level has all its nodes to the left side.



Properties of Binary Tree

- The maximum number of nodes on level i of a binary tree?
  - 2^i
- The maximum number of nodes in a binary tree of depth h?
  - $2^{h+1} - 1$
- The number of nodes in a complete binary tree with depth h ranges … ?
  - Between 2^h and $2^{h+1} - 1$
- The minimum number of levels (or minimum possible height) of a binary tree with n node?
  - $log_2(n+1) - 1$
    - Solve $n = 2^{h+1} - 1$

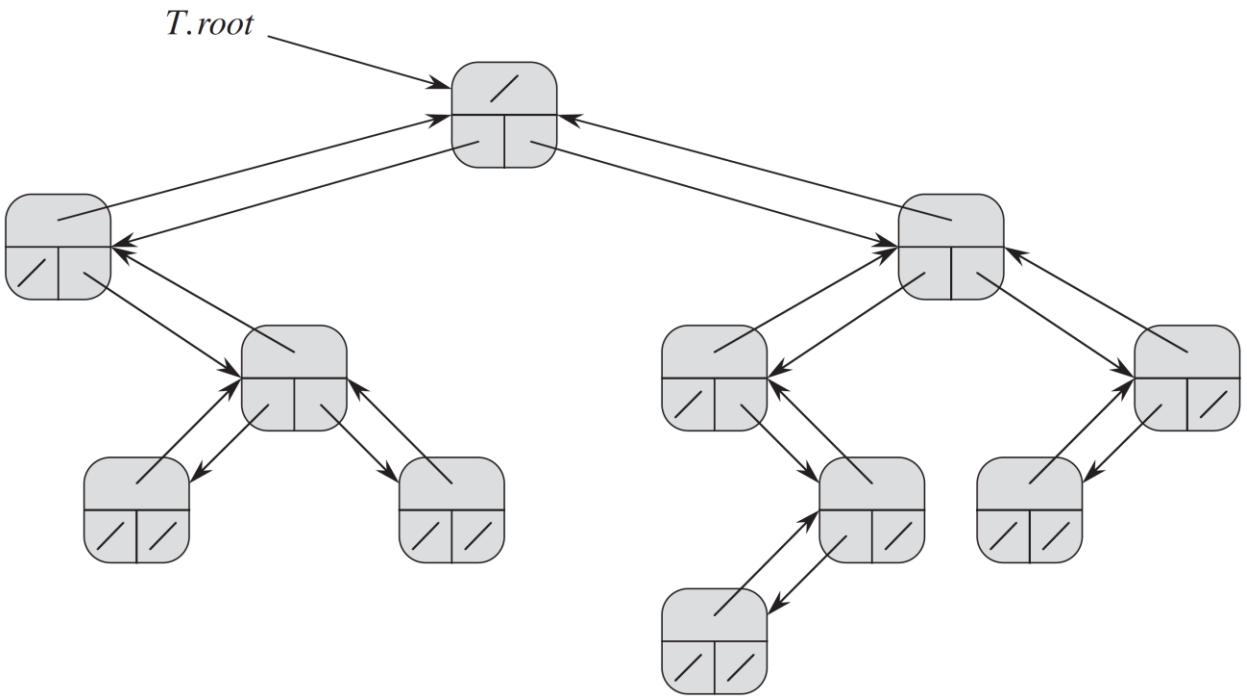
Operations of Binary Tree

- Create a one-node (or empty) binary tree
- Determine whether a binary tree is empty
- Determine what data is the binary tree's root

- What else…?

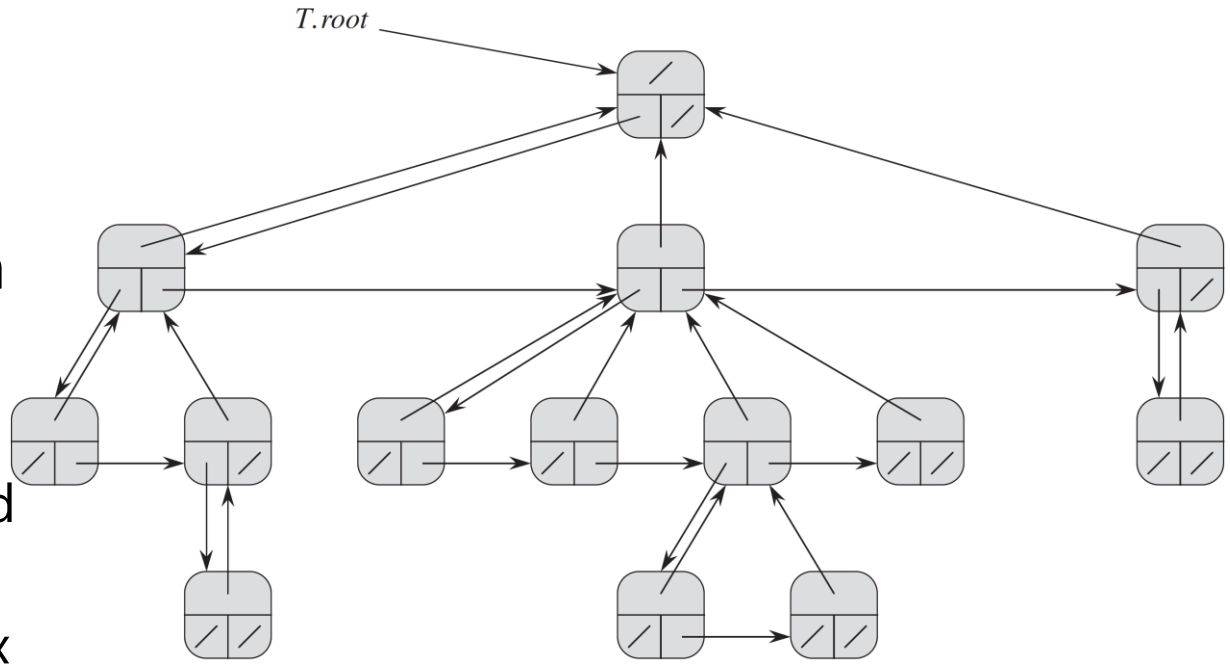
Representing Binary Trees

- Suppose that each node in a tree contains attributes
  - key
  - p: pointer to the parent
  - left: pointer to the left child
  - right: pointer to the right child
- If x.p = NULL, then x is the root
- If x has no left child, x.left = NULL
- If x has no right child, x.right = NULL
- The root of tree T is pointed by T.root.
- If T.root = NULL, the tree is empty

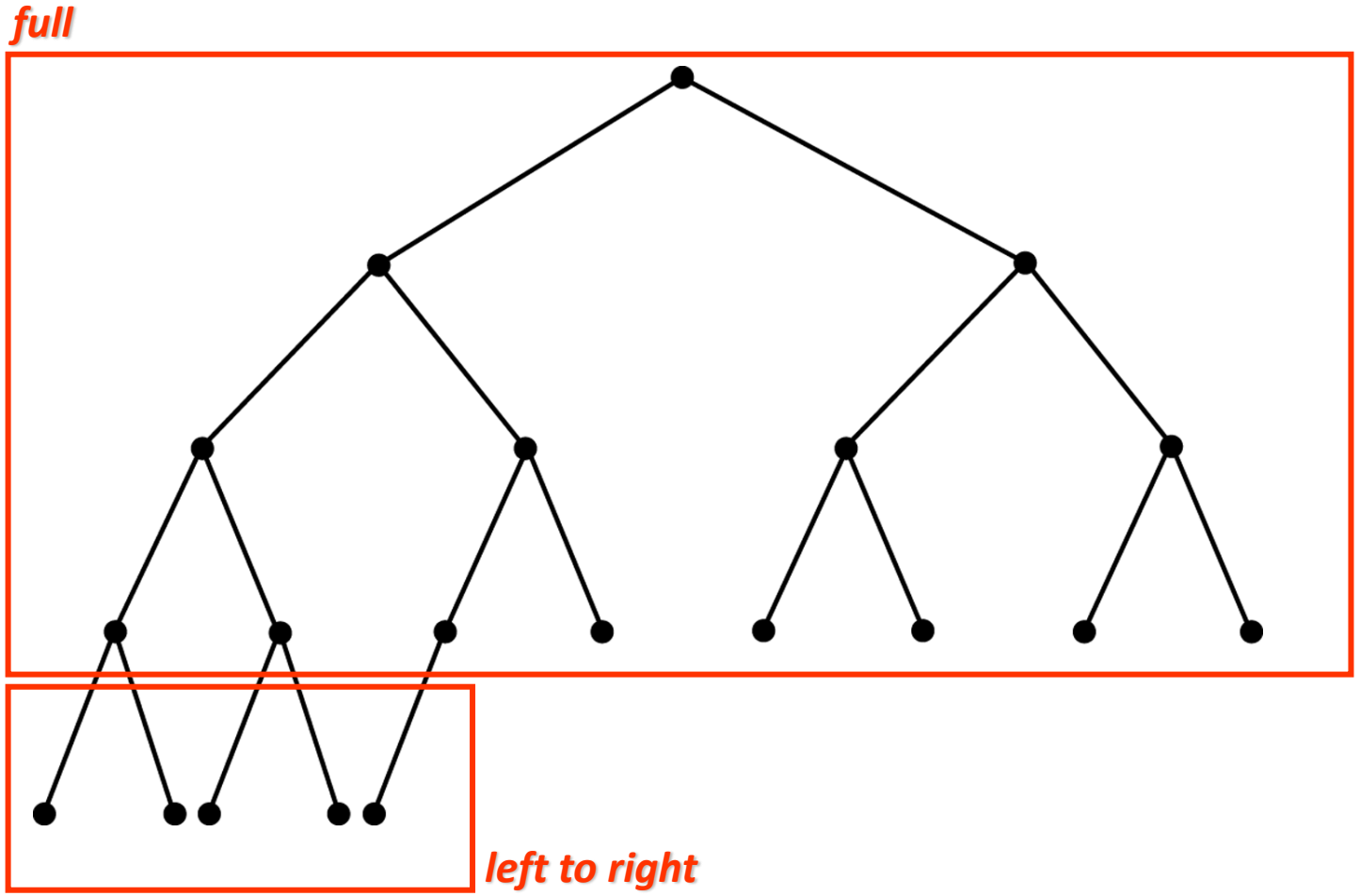


Representing Trees

- How can we represent unbounded number of children?
- Instead of having a pointer to each child, each node x has only two pointers:
  - x.leftChild points to the left most child of x.
  - x.rightSibling points to the sibling of x immediately to its right.
    - If x is the rightmost child of its parent, x.rightSibling = NULL.



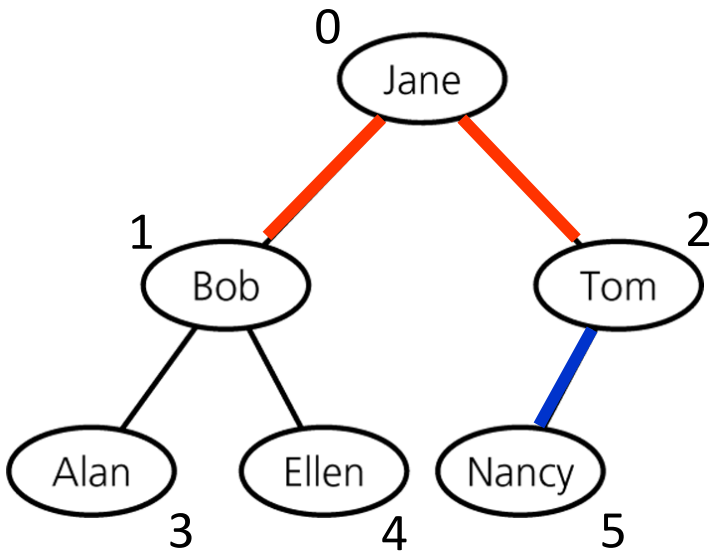
Complete Binary Trees



- A complete binary tree of height h is a binary tree full down to level h-1 with level h filled in from left to right

Representing Trees

- Complete binary tree can be represented by arrays and the indices.



tree

	item	leftChild	rightChild
0	Jane	1	2
1	Bob	3	4
2	Tom	5	–1
3	Alan	–1	–1
4	Ellen	–1	–1
5	Nancy	–1	–1
6	?	–1	7
7	?	–1	8
8	?	–1	9
.	.	.	.
.	.	.	.
.	.	.	.

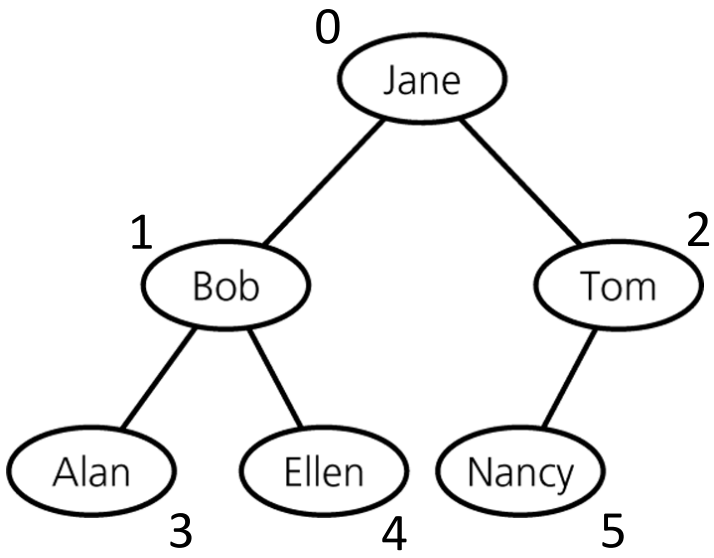
root: 0

free: 6

Free list

Representing Trees

- Complete binary tree (such as heap) can be represented by a single array and the index of the last node in the tree (heap).
- No links are needed!



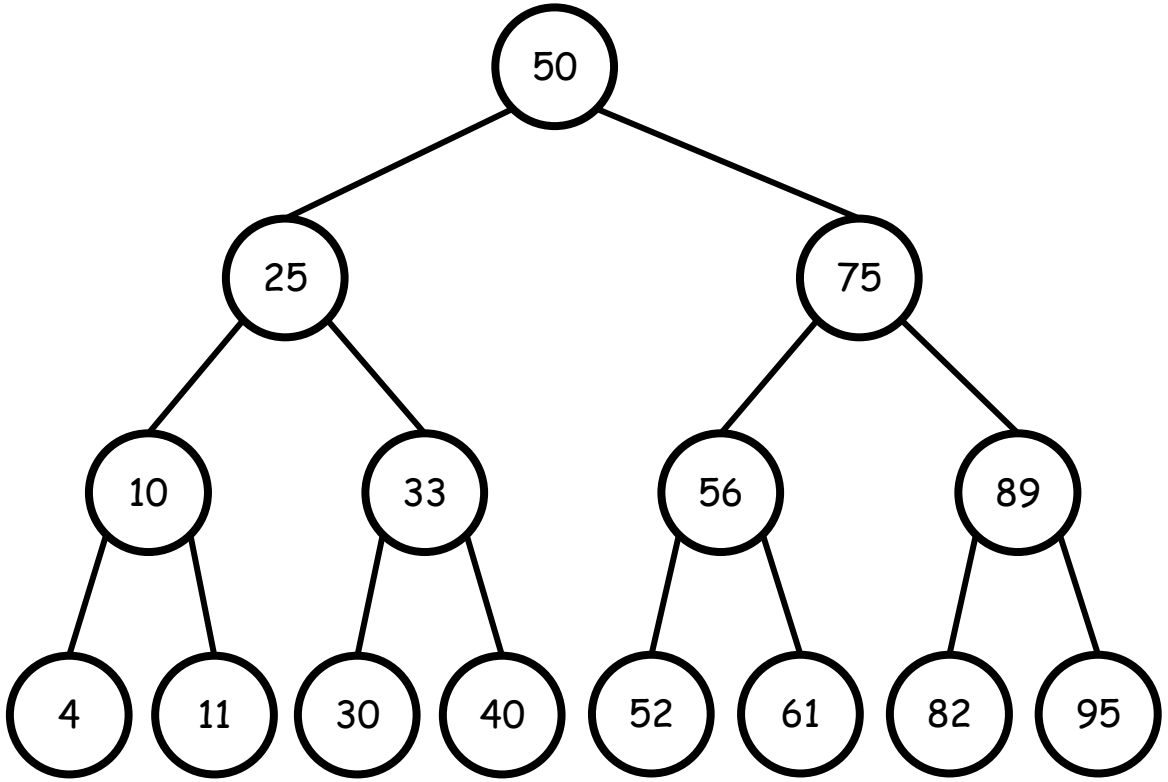
0	Jane
1	Bob
2	Tom
3	Alan
4	Ellen
5	Nancy
6	
7	

Binary Search Trees

Binary search tree is a **binary tree** where each internal node contains a key **larger than** all the keys in the node's left subtree and **less than** all the keys in the node's right subtree.

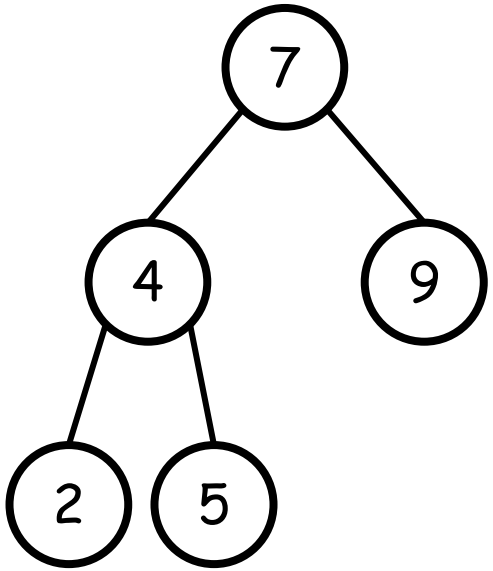
Binary Search Trees (BSTs)

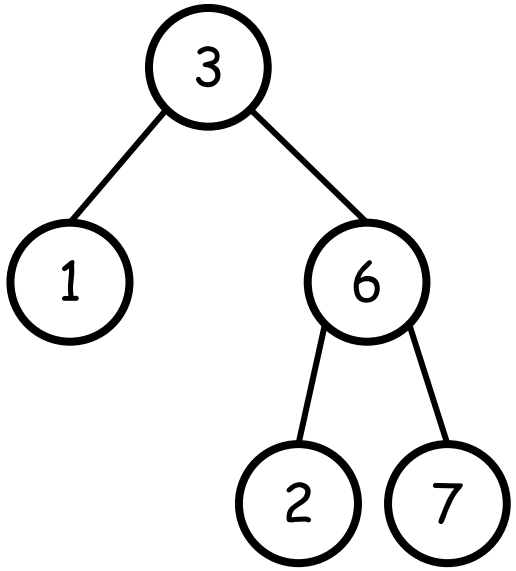
- For every node in a binary search tree,
- the values of all the keys in its **left** subtree are **smaller** than the node's key
- the values of all the keys in its **right** subtree are **larger** than the node's key



Binary Search Trees

- Are they binary search trees?





Usage of Binary Search Trees

- Set $r_1, r_2, \ldots, r_n$ of records associated with distinct key values $x_1, x_2, \ldots, x_n$, respectively.
- Given a search key x, find the record in a binary search tree T if it occurs in the set

Main Operations of Binary Search Trees

- Finding minimum/maximum in T
- **Searching** for x in T
  - Return either TRUE or FALSE, or return the pointer to the record
- **Inserting** a node with key x into T
  - If x already exists, do appropriate action (e.g., do nothing, return error message, increment reference count)
- **Deleting** a node with key x from T
  - If x does not exist, issue an error message

Finding Minimum

- Given root r in a BST, initially call FindMin(r)
- In BST, minimum element is the left-most node, which does not have the left child.

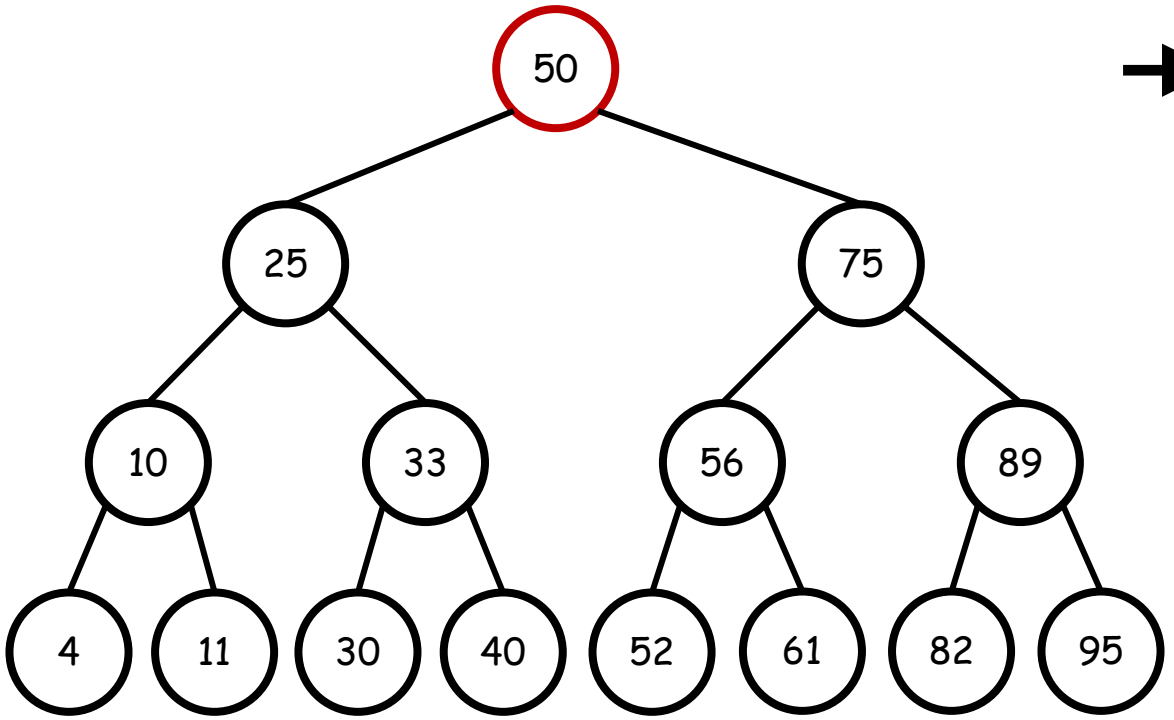
```
FindMin(p) // p: pointer to the root
    if p = NULL
        return NULL
    while p.left ≠ NULL
        p = p.left
    return p
```

Search

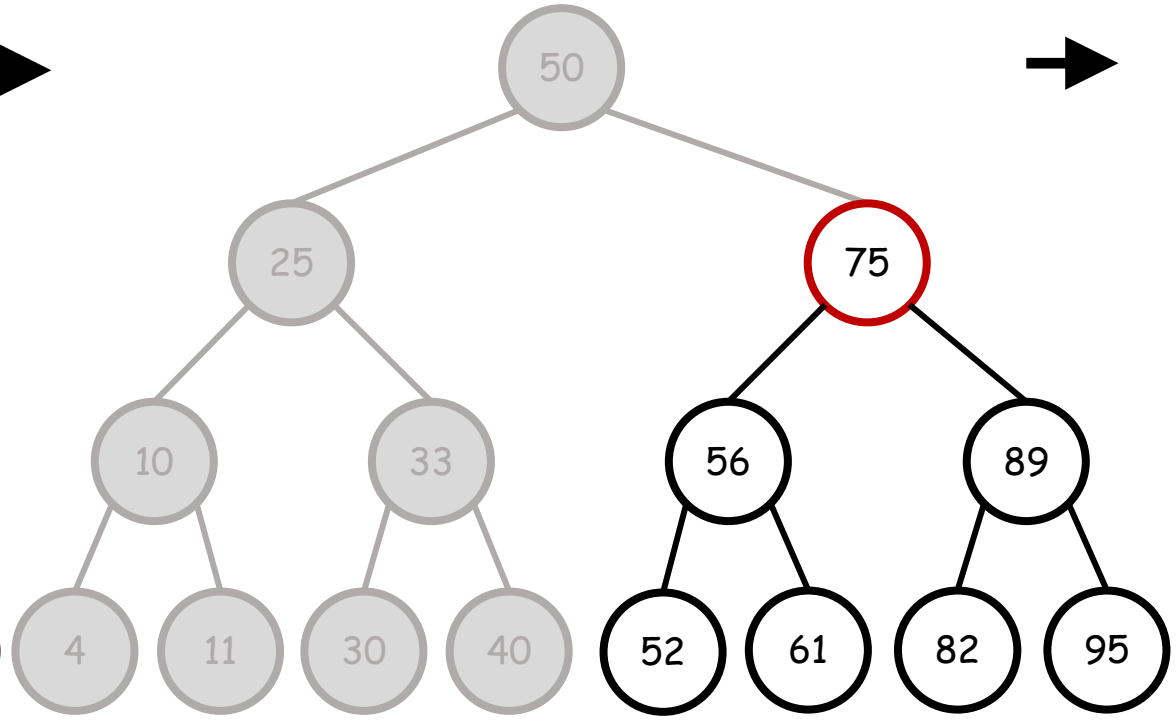
1. Designate a node to be the “current node.” (At the beginning of the algorithm, the root node is the first “current node.”)
2. Inspect the value at the current node.
   - If we’ve found the value we’re looking for, retrieve a pointer to the node
   - If the value we’re looking for is less than the current node, search for it in its left subtree.
   - If the value we’re looking for is greater than the current node, search for it in its right subtree.
3. Repeat 1 and 2 until we find the value we’re searching for, or until we hit the bottom of the tree (in which case our value must not be in the tree).

Finding 61

- Begin at the root.

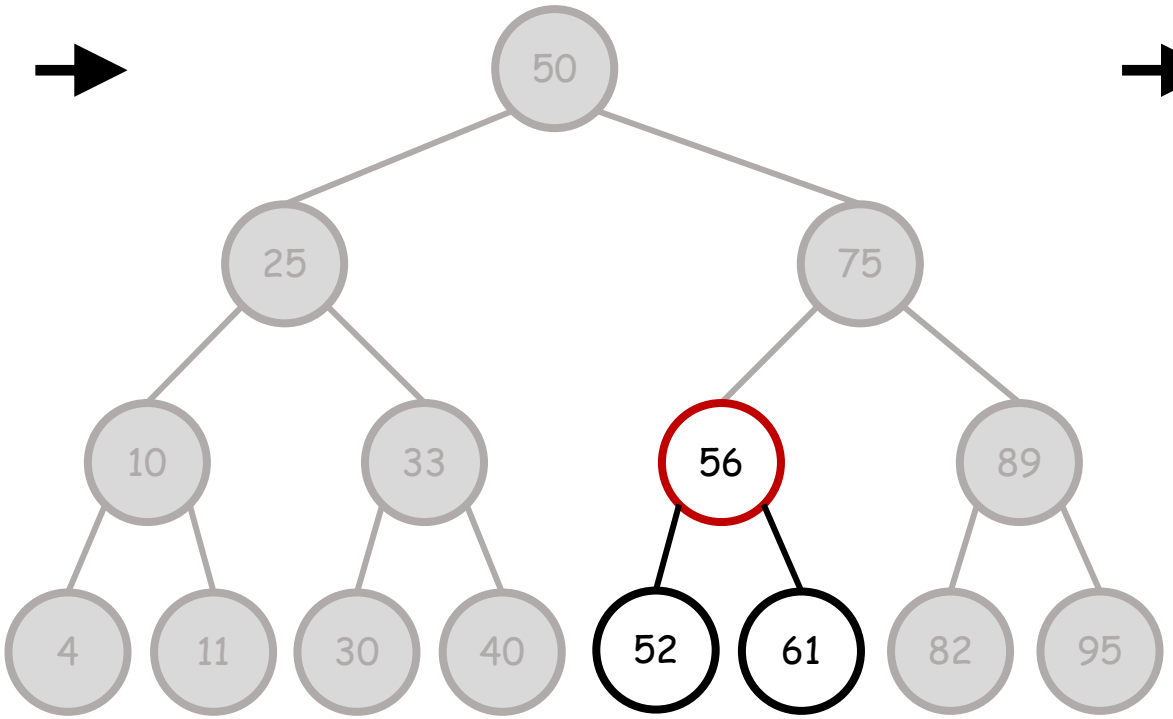


- Since 50 < 61, search the right child.

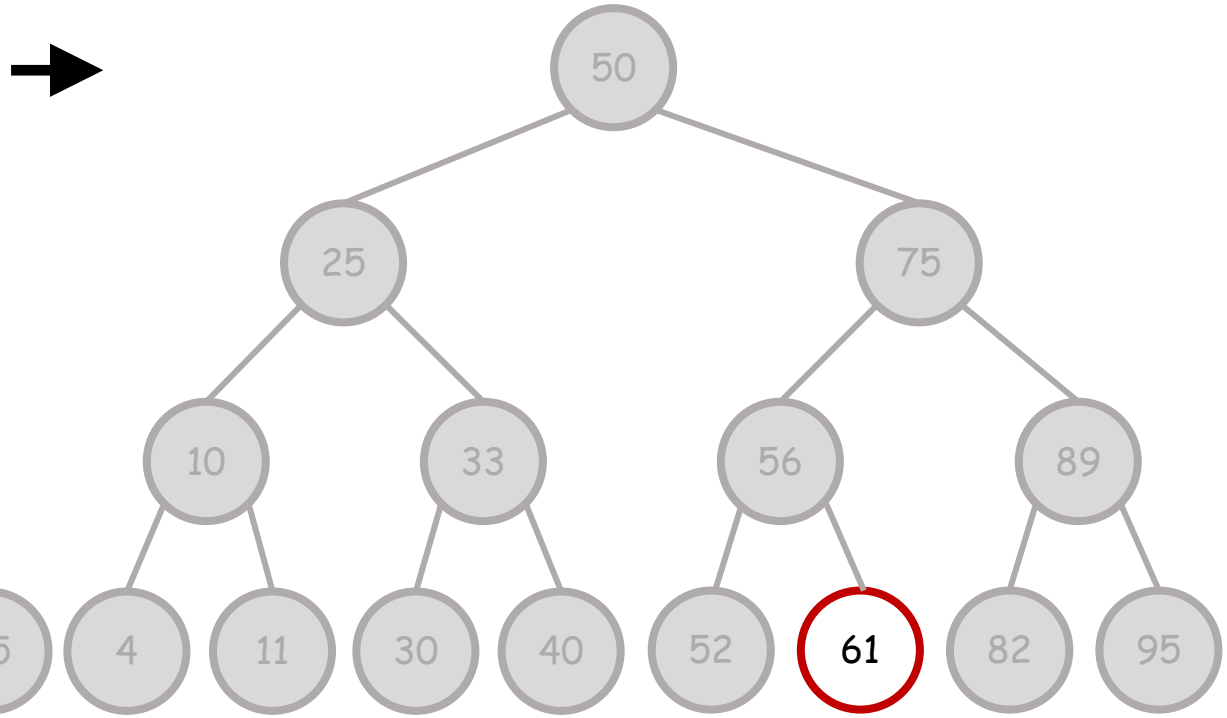


Finding 61

- Since 61 < 75, search the left child of 75.



- Since 56 < 61, search the right child of 56.



Search

- Recursive search in a binary search tree
- Given root r and key k to find, initially call Search(r, k)

```
Search(p, k) // p: pointer to the current node, k: key to find
    if k = p.key
        return p // successful search
    else if k < p.key and p.left ≠ NULL
        return Search(p.left, k) // recur on the left subtree
    else if k > p.key and p.right ≠ NULL
        return Search(p.right, k) // recur on the right subtree
    return p // unsuccessful search
```

Insertion

1. Search for key k (Search(T.root, k))
2. If found, that item's existing value is reassigned (or do nothing)
3. Otherwise, a node for the new item can be inserted into the tree T in place of the empty subtree that was reached at the end of the failed search.

```
Insert(k) // k: key to insert
    p = Search(T.root, k)
    if k < p.key
        add node with key k as left child of p
    else if k > p.key
        add node with key k as right child of p
    // else, that is, k = p.key, and k already exists in the tree
    //    do nothing
```

Insertion (Recursive)

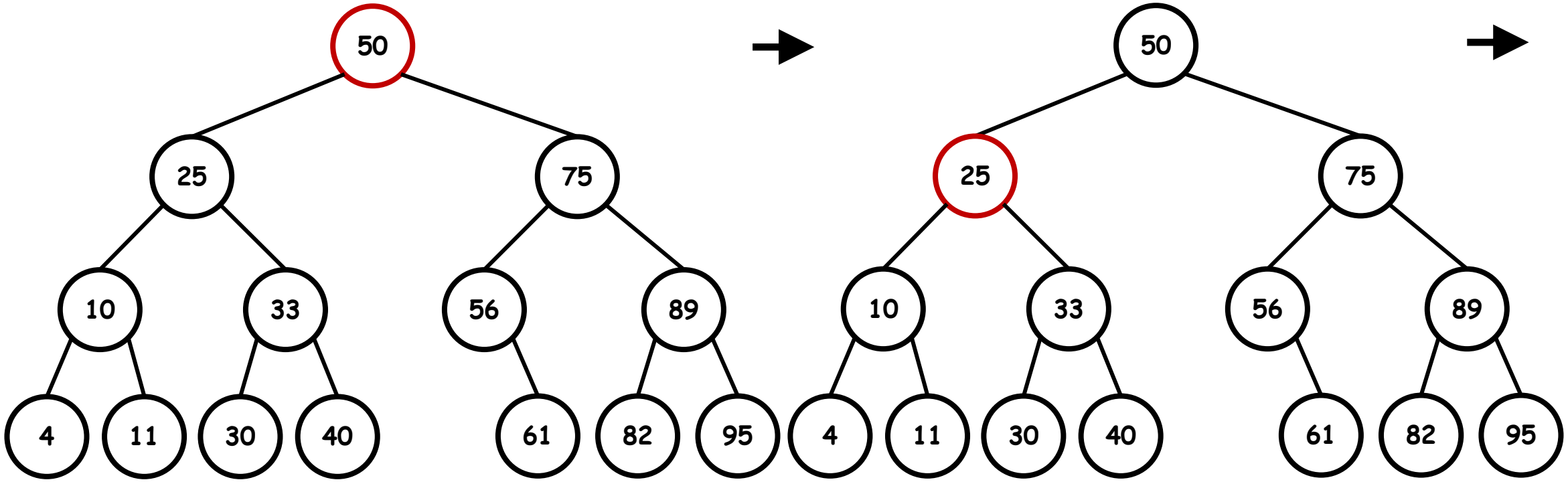
- First, we call Insert(T.root, key)
- Compare the root of the current subtree with the given key k until finding the right position

```
Insert(root, k) // root: the root of current subtree, k: key to insert
    if root = NULL
        root.key ← k
        root.left ← root.right ← NULL
    else if k < root.key
        root.left ← Insert(root.left, k)
    else if k > root.key
        root.right ← Insert(root.right, k)
    // else, that is, if k = p.key
    //    do nothing
    return root  // recursive function, so do not forget this line!
```

Inserting 45

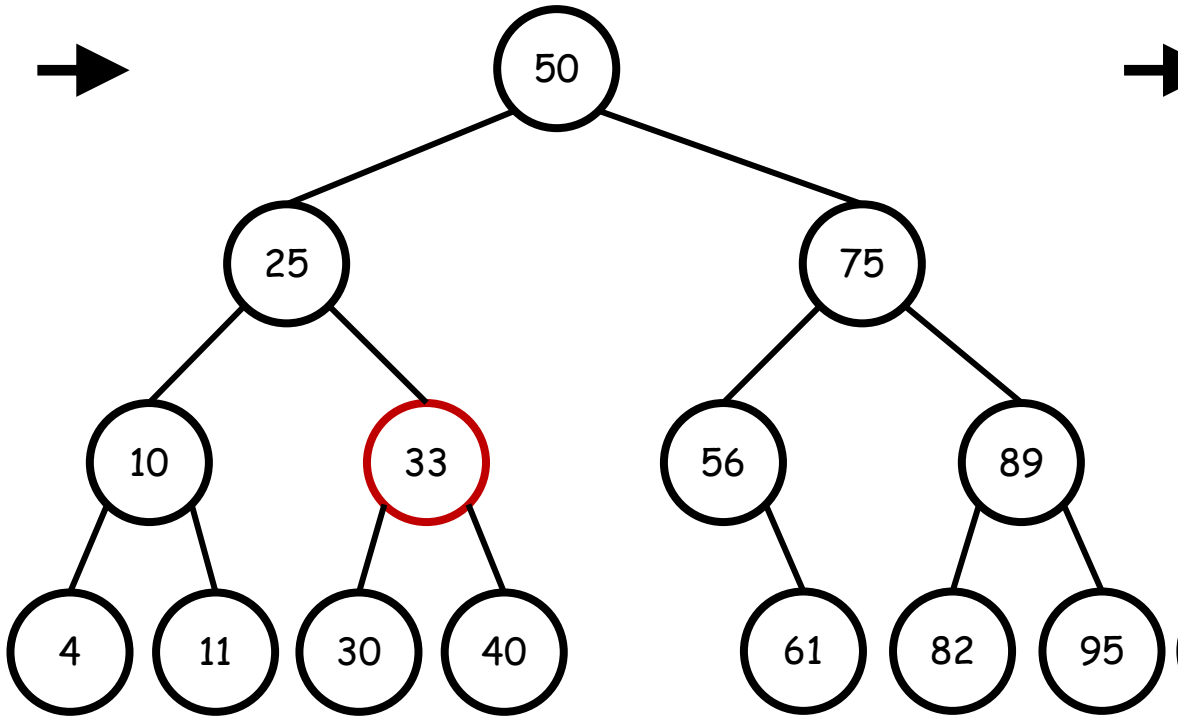
- First, find the correct node to attach 45 to.
- Start at the root.

- Since 45 < 50, drill down to the left child.

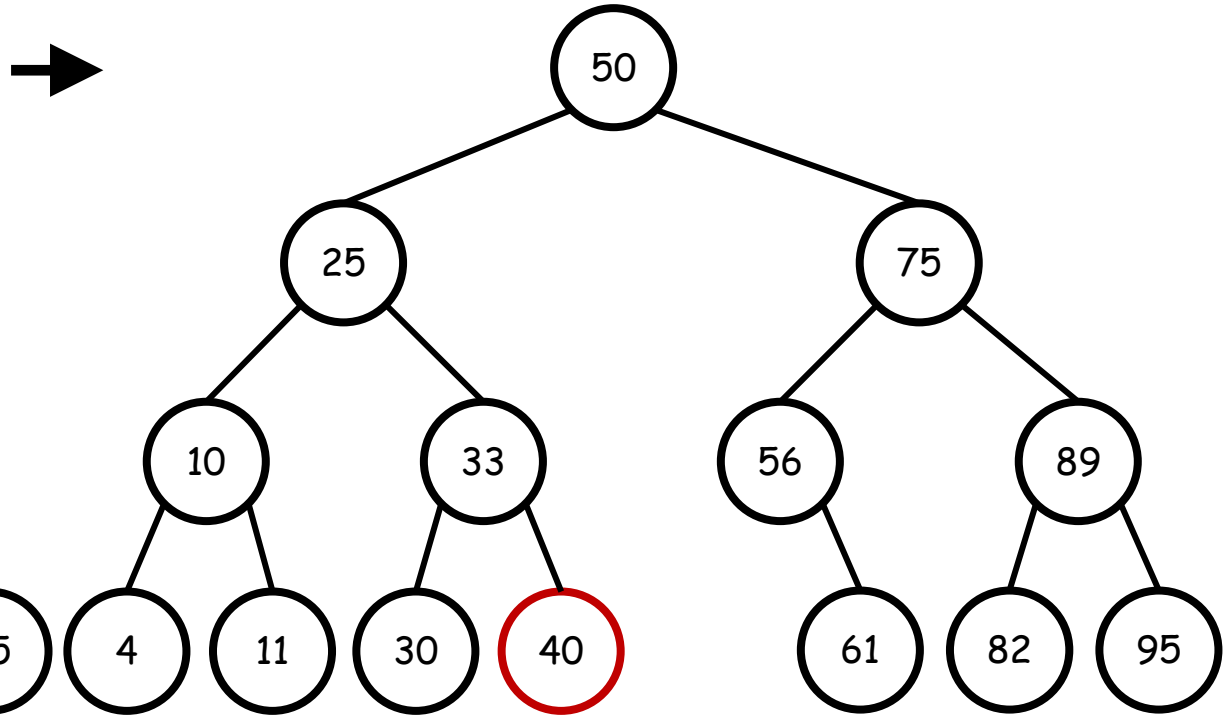


Inserting 45

- Since 45 > 25, inspect the right child.

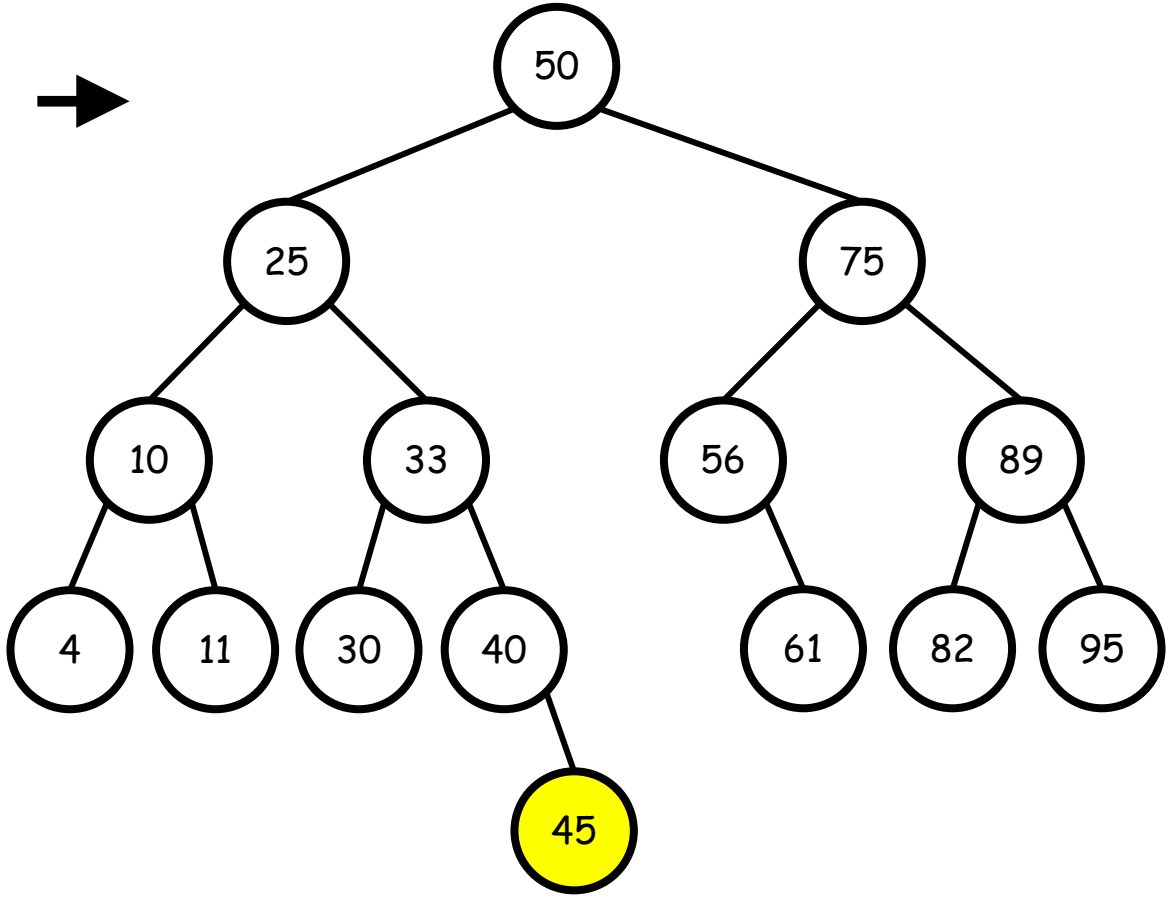


- Since 45 > 33, drill down to 33's right child.
- We've reached a node that has no children.



Inserting 45

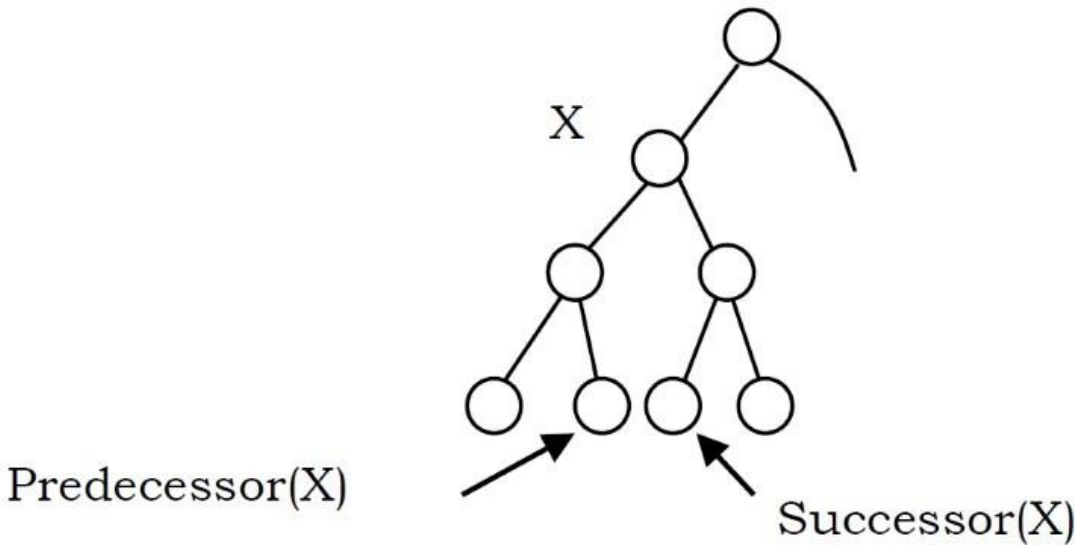
- Since 45 > 40, insert 45 as a right child of 40.



- Insertion takes O(logn) time on average.
- This is what make BST so efficient.

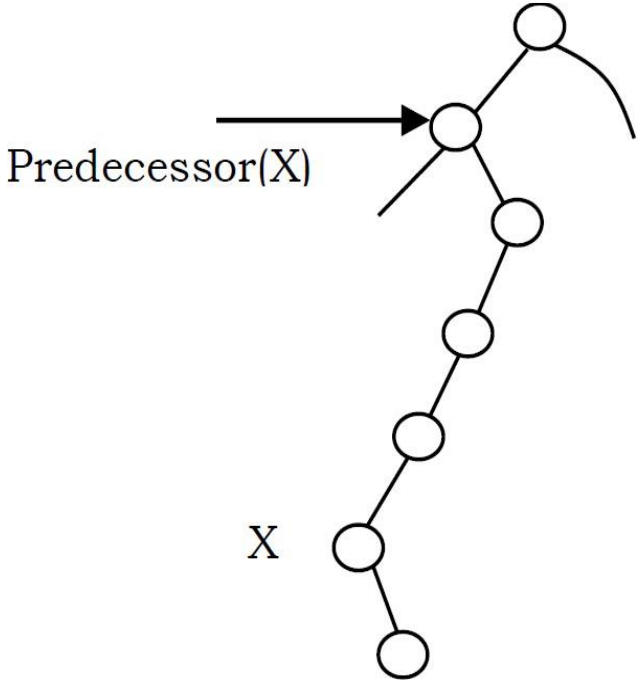
Predecessor and Successor

- Suppose that all keys are distinct.
- If x has two children,
- Predecessor of x is the maximum value in its left subtree
- Successor of x is the minimum value in its right subtree



Predecessor and Successor

- If x does not have a left child,
- Predecessor of x is its first left ancestor.

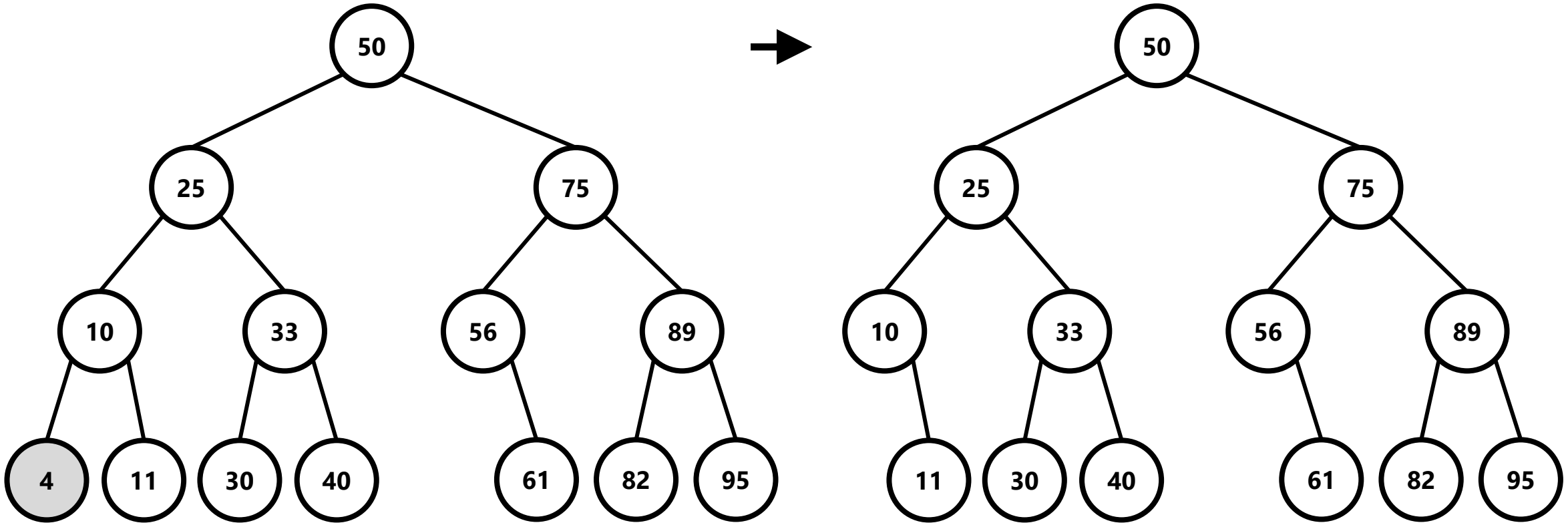


Deletion

1. Run **Search(T.root, k)** to find the node of T storing key equal to k.
2. If the node has no children, delete it.
3. If the node has at most one child, delete the node and plug the child into the spot where the deleted node was.
4. If the node has two children,
   i. Replace the node with its successor node.
      - (Because the node has two children, the successor is the leftmost position of the right subtree of the node.)
   ii. If the successor node has a right child, take the former right child of the successor node and turn it into the left child of the former parent of the successor node.

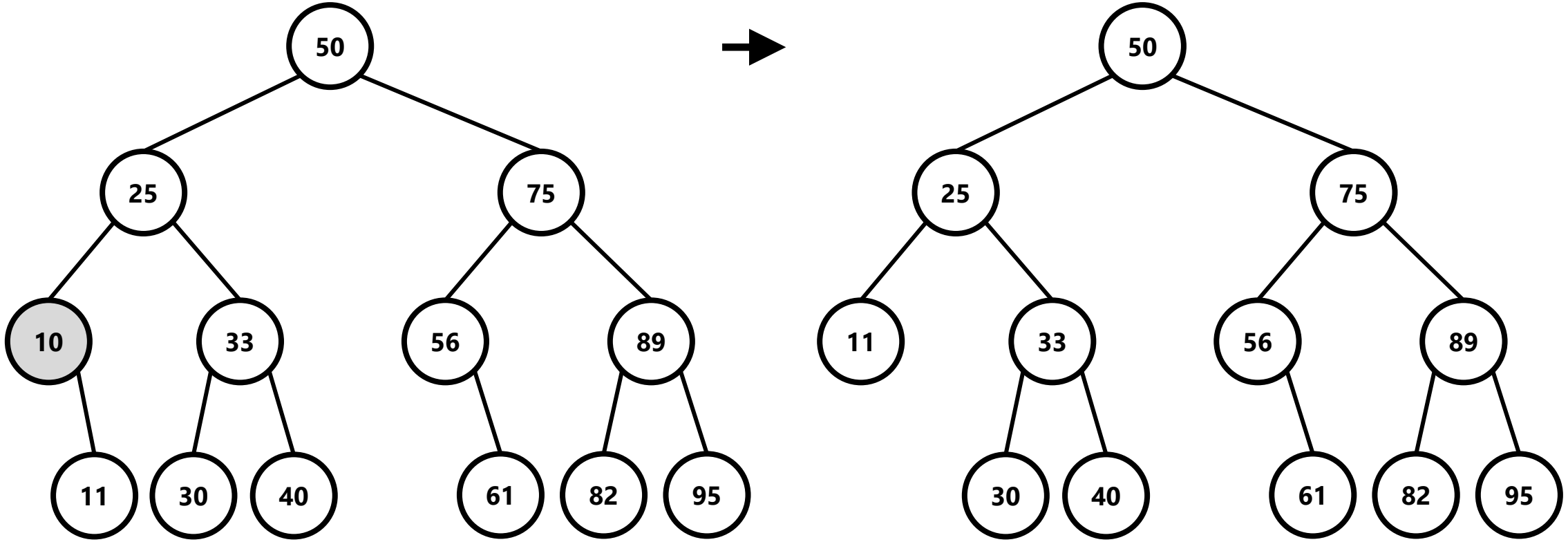
Deleting 4

- If the node has no children, delete it.



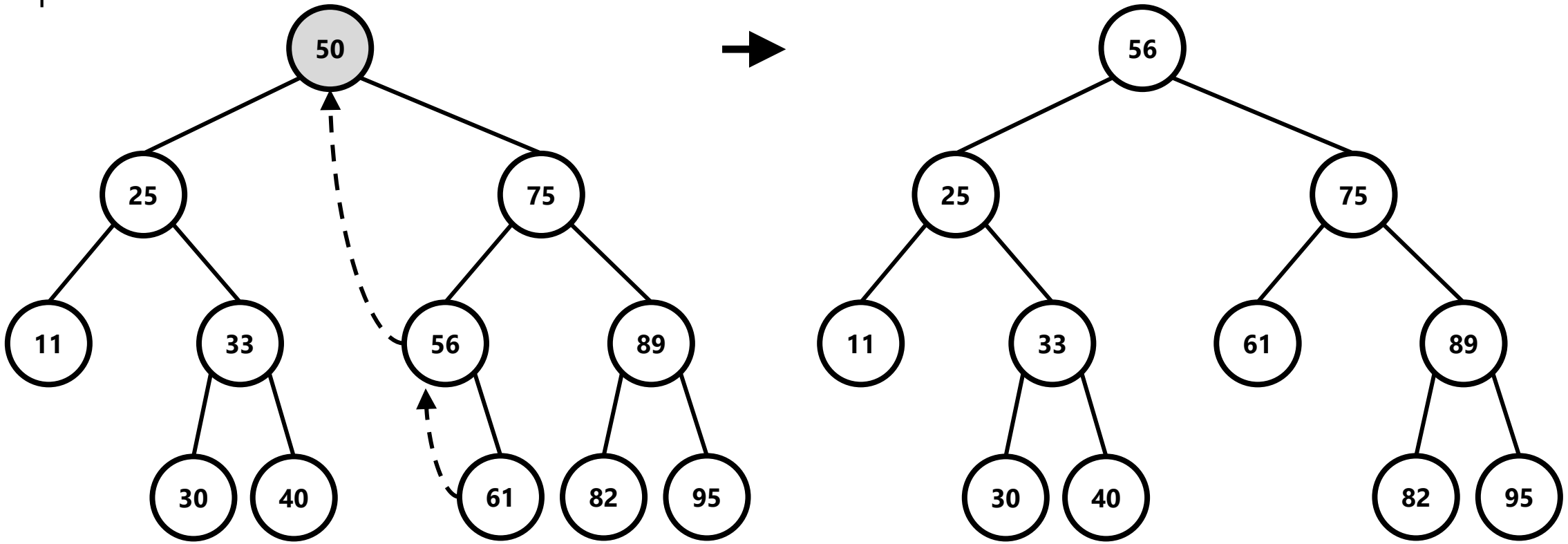
Deleting 10

- If the node has one child, delete the node and plug the child into the spot where the deleted node was.



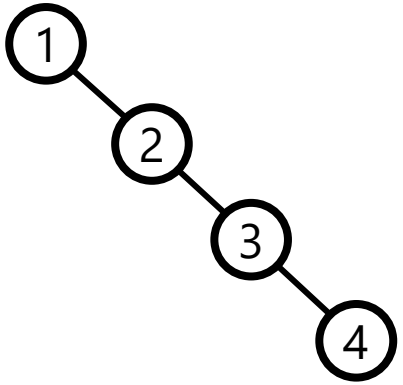
Deleting 50

- If the node has two children, replace the node with its successor node.
- If the successor node has a right child, turn the right child into the left child of the former parent of the successor node.

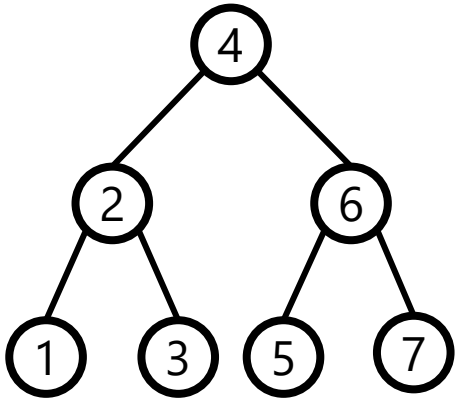


Analysis of Binary Search Trees

- Runtime of search, insertion, deletion is proportional to the height of the tree.
- What is the height of the tree with n nodes?
  - Worst case: $O(n)$ (skew tree)
  - Best case: $O(logn)$ (complete binary tree)



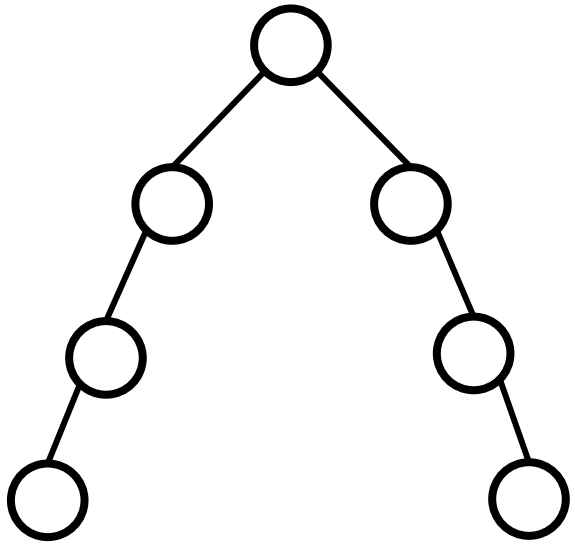
Worst case: insert 1, 2, 3, 4, …

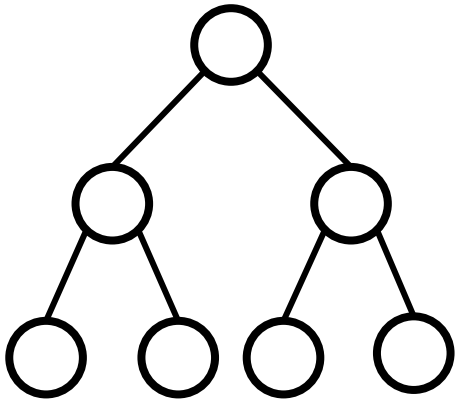


Best case: insert 4, 2, 6, 1, 3, 5, 7

Balanced Binary Trees

- Unbalanced trees can lead to the worst case.
- Can we rebalance the tree so that tree always has the O(logn) height?



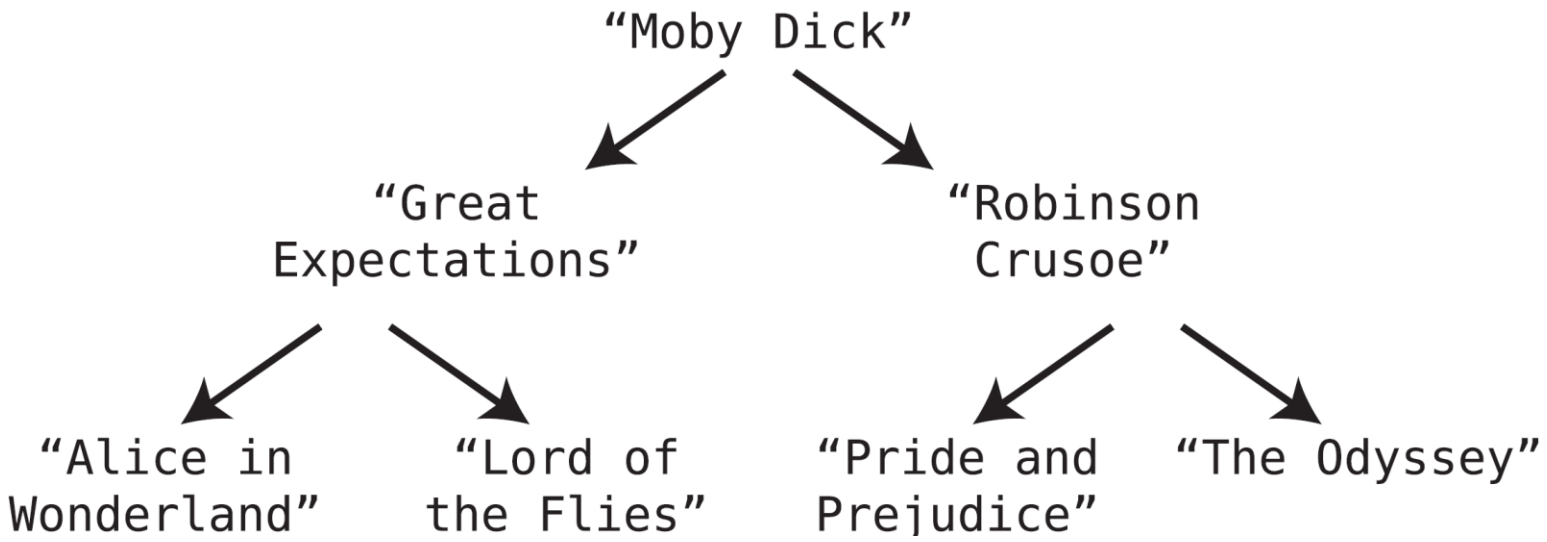


Tree Traversal

An algorithm that visits every node in a tree.

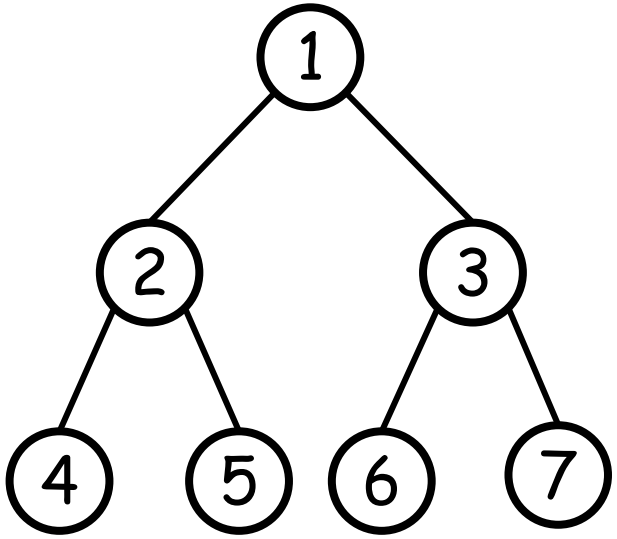
Binary Search Tree Traversal

- Print the entire list of keys (or book titles) in a particular order.
  - Visit every single node in the tree
  - Traverse tree in a particular order (alphabetically ascending order)



Tree Traversals

- The classification of tree traversals is based on the order in which current node is processed.
  - Level Order Traversal
    - 1-2-3-4-5-6-7
  - Preorder Traversal
    - 1-2-4-5-3-6-7
  - Inorder Traversal
    - 4-2-5-1-6-3-7
  - Postorder Traversal
    - 4-5-2-6-7-3-1



Level Order Traversal

- This method is inspired from breadth first traversal in a graph.

```
LevelOrder(root)
    Q ← empty queue
    Q.enqueue(root) // insert
    while (Q is not empty) do
        v ← Q.dequeue() // delete v from Q (v is the oldest entry in the queue.)
        Mark v as visited
        for each child c of v do
            Q.enqueue(c) // insert v's children to the end of the queue
```

Preorder Traversal

- For BST, each node is processed before (pre) either of its subtrees.

1. Visit the node.
2. Traverse the left subtree in preorder
3. Traverse the right subtree in preorder.

Preorder(x) // general tree
  if x ≠ NULL
  Mark x as visited
  for each child c of x **do**
  Preorder(c)
	Preorder(x) // binary tree
	  if x ≠ NULL
	  Mark x as visited
	  Preorder(x.left)
	  Preorder(x.right)

Postorder Traversal

- For BST, each node is visited after both subtrees.

1. Traverse the left subtree in postorder.
2. Traverse the right subtree in postorder.
3. Visit that node.

Postorder(x) // general tree
  if x ≠ NULL
  for each child c of x **do**
  Postorder(c)
  Mark x as visited
	Preorder(x) // binary tree
	  if x ≠ NULL
	  Postorder(x.left)
	  Postorder(x.right)
	  Mark x as visited

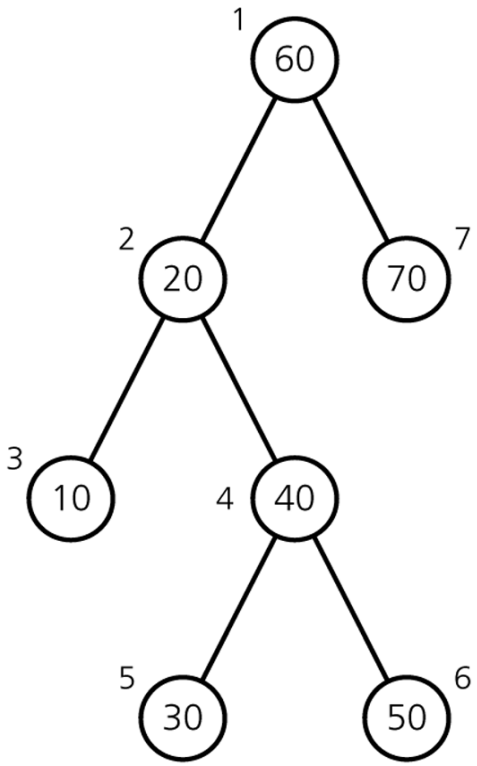
Inorder Traversal

- In BST, each node is visited between the subtrees.

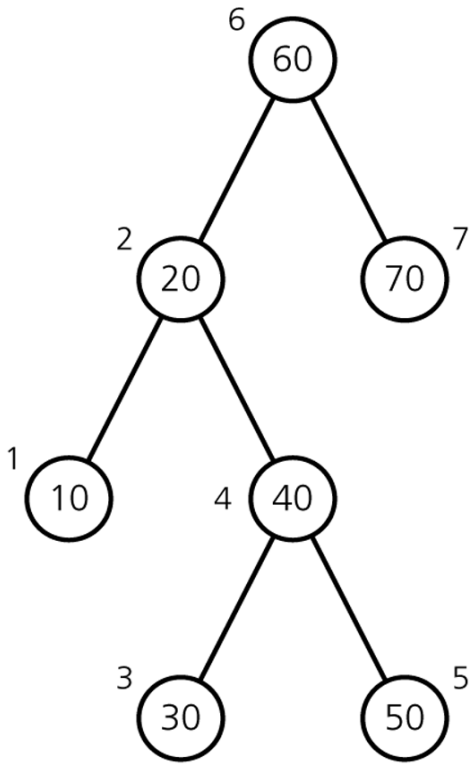
1. Traverse the left subtree in inorder.
2. Visit that node.
3. Traverse the right subtree in inorder.

```
Inorder(x) // for only binary tree
    if x ≠ NULL
        Inorder(x.left)
        Mark x as visited
        Inorder(x.right)
```

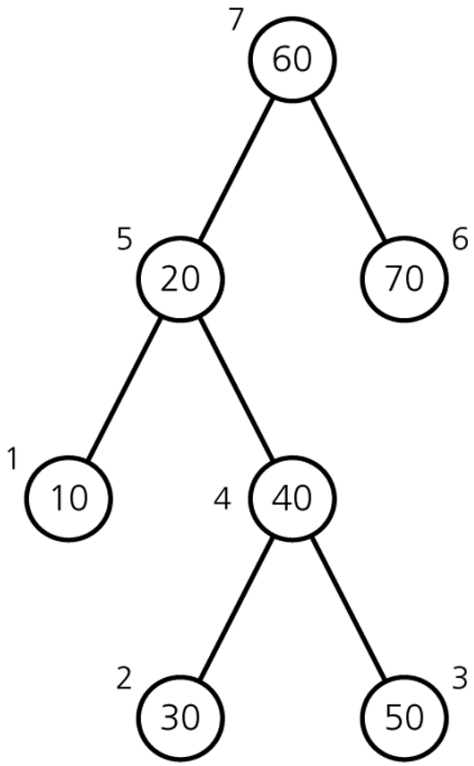
Traversals in Binary Search Trees



(a) Preorder: 60, 20, 10, 40, 30, 50, 70



(b) Inorder: 10, 20, 30, 40, 50, 60, 70



(c) Postorder: 10, 30, 50, 40, 20, 70, 60

(Numbers beside nodes indicate traversal order.)

Efficiency of Binary Search Trees

- Suppose we are given a BST with n nodes.

Operation	Average Case	Worst Case
FindMin/FindMax	O(logn)	O(n)
Search	O(logn)	O(n)
Insertion	O(logn)	O(n)
Deletion	O(logn)	O(n)
Traversal	O(n)	O(n)